

Overview

🎯 In this course, you will explore how to utilize Behavior-Driven Development (BDD) within Squish to perform tests on your applications based on outlined criteria at the beginning of the development cycle.

👤 This course is for software testers, QA engineers, and developers interested in automated testing interested in using Squish for behavior-driven testing.

👉 Before getting started, we recommend taking a look at our Squish - Test Synchronization and Data-Driven Testing courses.

[Behavior Driven Testing | Squish 9.2](#)

[Squish for Qt Tutorials | Squish 9.2](#)

[GitHub - qt-learning/Behavior-Driven-Development · GitHub](#)

What Will I Learn ?

1. How to create an structure behaviour-driven test cases in Squish using Gherkin syntax
 2. How to execute behaviour-driven test cases within Squish using shared and externally imported data.
 3. How to migrate to and debug behaviour-driven tests.
-

Introduction

BDD is an extension of Test-Driven Development (TDD) that places definition of acceptance criteria at the beginning of the development process as opposed to writing tests after the software has been developed. With this approach, code changes are cyclically done based on the test results.

BDD tests consists of:

- **Feature** files which describe the product features (or requirements) through the expected application behaviour described as Scenarios (E.g.,)
- **Scenarios** are a sequence of Steps (E.g.,)
- **Steps** represent the actions or verifications needed to be tested in the Scenarios (E.g.,)

BDD focuses on expected application behaviour, not on implementation details, and its tests are described in a human-readable Domain Specific Language (DSL) .

A benefit of using DSL in BDD is that tests can be created by product owners, testers, or business analysts because technical language is not a requirement. Moreover, these tests can be used as evolving documentation during product development.

In Squish, BDD tests are created using the Gherkin syntax. Most of a Feature file can be written in any language and in any form but:

→ A feature / Scenario structure must be used.

→ Each line starts with one of the Gherkin keywords which are fixed. Common keywords are:

- **Feature** : must be the first keyword in a Gherkin document
- **Scenario**: (or Example:) is the keyword to use before the sentence that illustrates a business rule. The scenario consists of a list of steps.
- **Given, When, Then, And** : keywords used before a sentence describing a step
 - **Given** = for step that describes an initial context
 - **When** = for a step that describes an event.
 - **Then** = for a step that describes an expected outcome.
 - **And** = used with succession Given or Then steps.
- **Scenario Outline**: the keyword used to run the same Scenario multiple times, with different combinations of values.
- **Examples**: used with Scenario Outline. Precedes the combinations of values used in the Scenario Outline.

BDD tests written in DSL can be turned into executable tests in Squish.

This is the syntax for a Feature. It describes the actions of maintaining entries in an address book application, and contains a scenario where the user removes initial entries and adds a new one.

None

Feature: Filling of quickaddressbook

As a user I want to fill the Address Book with entries

#Steps for new entry

Scenario: State after adding one entry

Given quickaddressbook application is running

When I remove initial entries

And I add a new person 'John', 'Doe', '500600700', 'john@m.com' to address book

Then '1' entries should be present

Creation of a BDD Test

Squish supports two types of Test Cases, "Script Test Case" and "BDD Test Case", which can exist in the same test suite. A new BDD test case is created by clicking on New Test Case button > New BDD Test Case.

A default BDD test case consists of :

- A test (.py,.js, ...) based on the scripting language used in the test suite) that drives the execution of the text.
- A test.feature feature file written in DSL.
- Step implementations. Shared by the entire test suite and located in shared/steps/steps (.py, .js, ...)

First, the **test.feature** file must be prepared. Once it is ready, the code implementation corresponding to the steps needs to be written. This can be done manually by editing the steps file or using the recording feature of Squish.

Like the recording of a script test case, Squish will start the AUT (Application Under Test) and open the Control Bar. In addition, the Control Bar will display the list of steps for each scenario. If the step has already been implemented in code, Squish will run it. Otherwise, Squish will begin to record a step. While recording a step, all interactions with the AUT and verification points will be recorded. When one step is recorded, it is possible to move to the next one by clicking on the Finish Recording Step. For steps that have been recorded, Squish will generate the step definitions in steps.

In the steps file, a step() function is generated for each step recorded. Step functions can also be written manually. Each step() function has a context singleton object passed in as the first argument. It is used to store text, table, or generic data between the steps. In Python, the implementation is preceded by a decorator, which defines the type and name of the step:

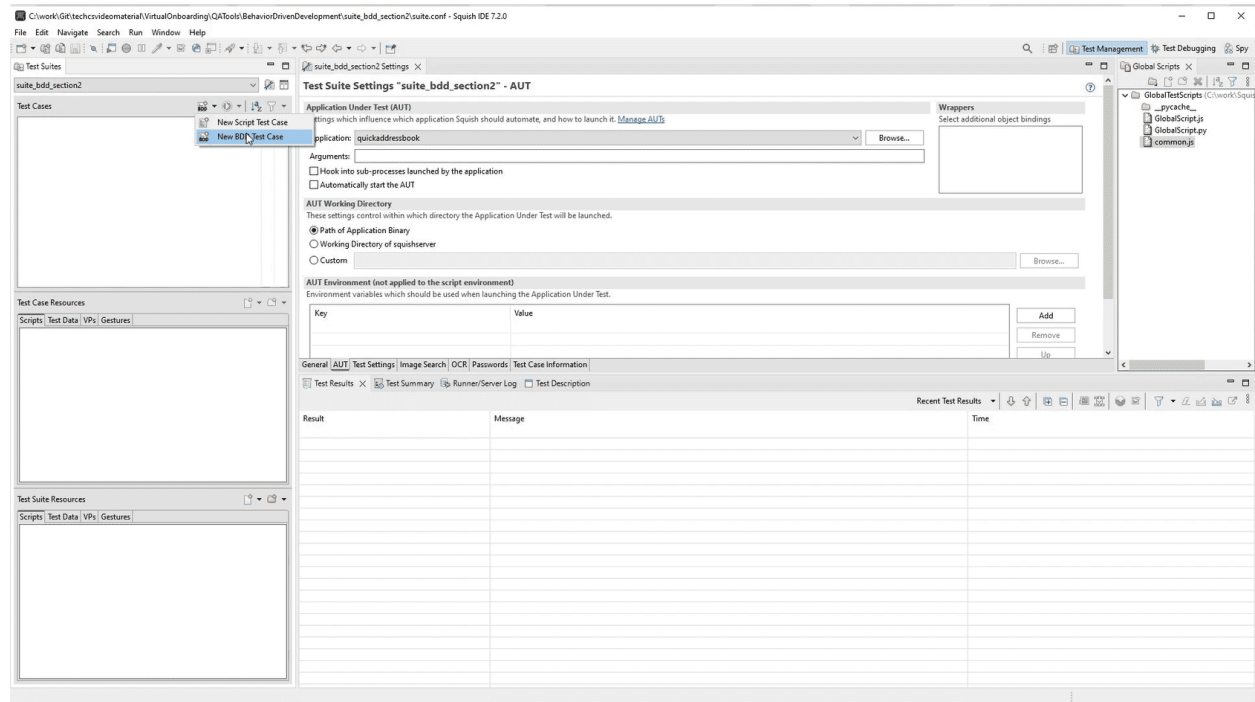
None

```
@Given("quickaddressbook application is running")
def step(context):
    startApplication("quickaddressbook")
    test.compare(waitForObjectExists(names.mainWindow).visible,
True)
```

Except for these particularities, the step() functions' contents follows the same syntax and uses the same API as those used in standard test scripts.

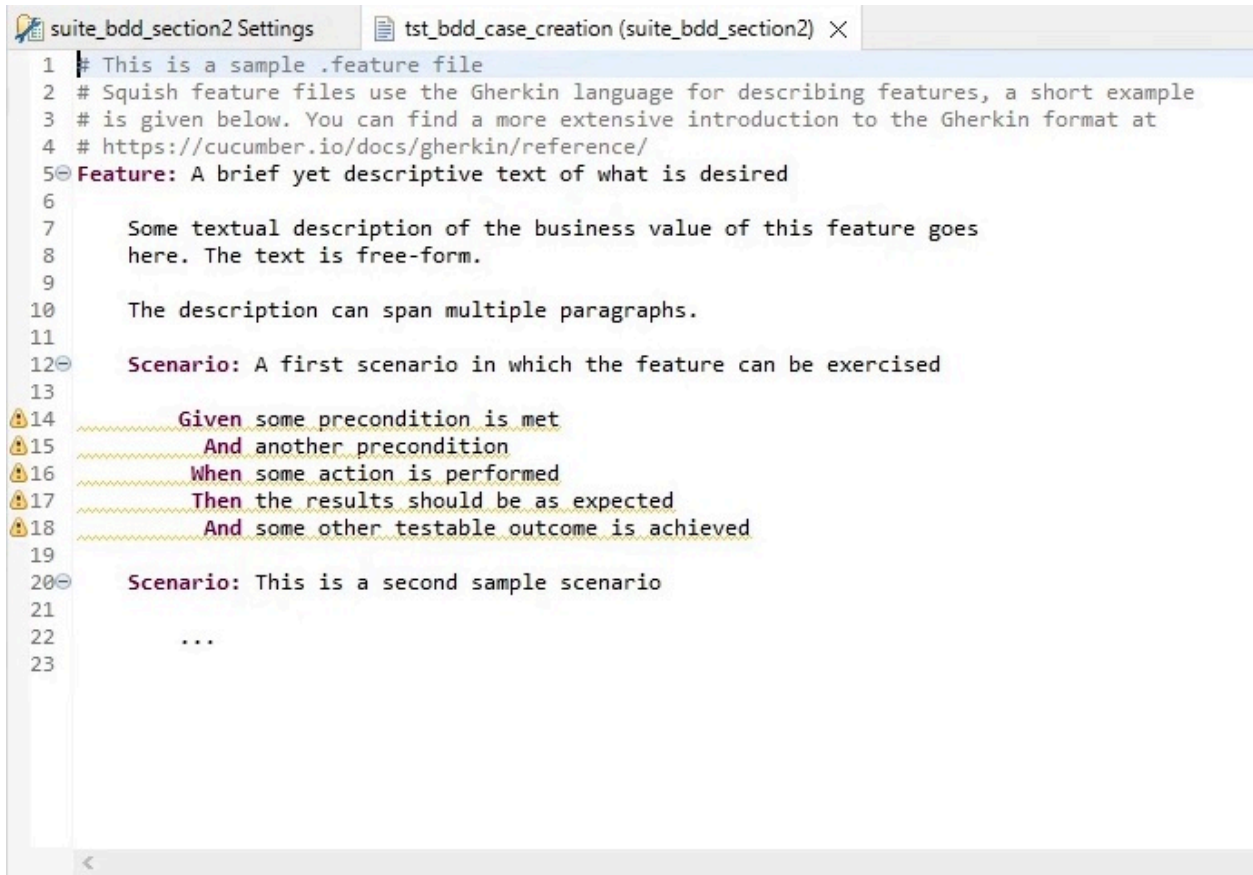
STEPS - How to Create and Record a BDD Test Case in the Squish IDE

Step 1: Create a new BDD test case



Using the Qt Quick Address Book example found in the examples subdirectory in the Squish installation, we create a new BDD test case in Squish, resulting in the generation of a test.py file and a test.feature file within the Test Case Resources.

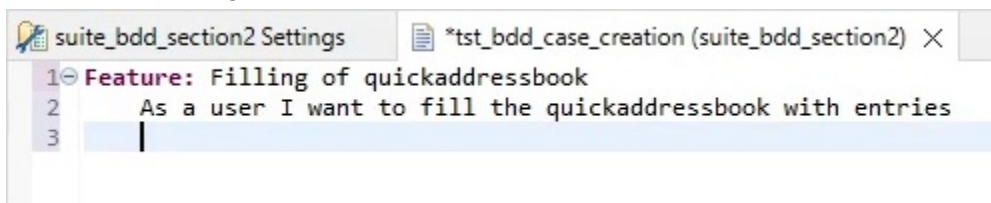
Step 2: Modifying the feature file



```
1 # This is a sample .feature file
2 # Squish feature files use the Gherkin language for describing features, a short example
3 # is given below. You can find a more extensive introduction to the Gherkin format at
4 # https://cucumber.io/docs/gherkin/reference/
5 Feature: A brief yet descriptive text of what is desired
6
7     Some textual description of the business value of this feature goes
8     here. The text is free-form.
9
10    The description can span multiple paragraphs.
11
12 Scenario: A first scenario in which the feature can be exercised
13
14     Given some precondition is met
15     And another precondition
16     When some action is performed
17     Then the results should be as expected
18     And some other testable outcome is achieved
19
20 Scenario: This is a second sample scenario
21
22     ...
23
```

Before recording, we need to modify the test.feature file to describe the test scenario. We detail our test by writing scenarios and corresponding steps in Gherkin syntax. Let's start by clearing the template and write our test.

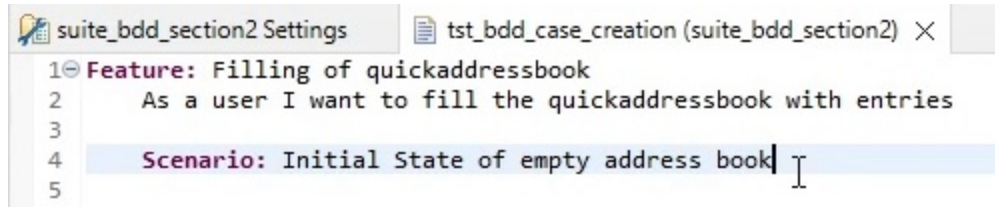
Step 3: Describing the feature to test



```
1 Feature: Filling of quickaddressbook
2     As a user I want to fill the quickaddressbook with entries
3
```

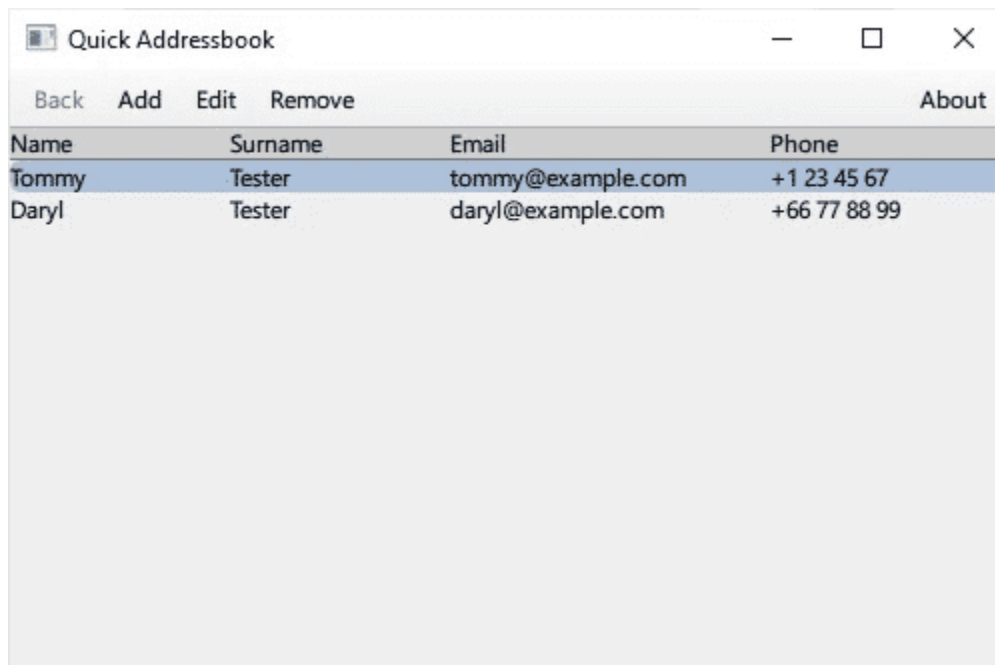
Let's start to write our test.feature file by describing the software feature used in this test and adding a description. Remember, we can right-click in the file and select "Format as a Gherkin file".

Step 4: Describing the scenario



We detail our test by writing scenarios and corresponding steps in Gherkin syntax. In the example, we want to check the initial state of the empty address book when there is no entry in it.

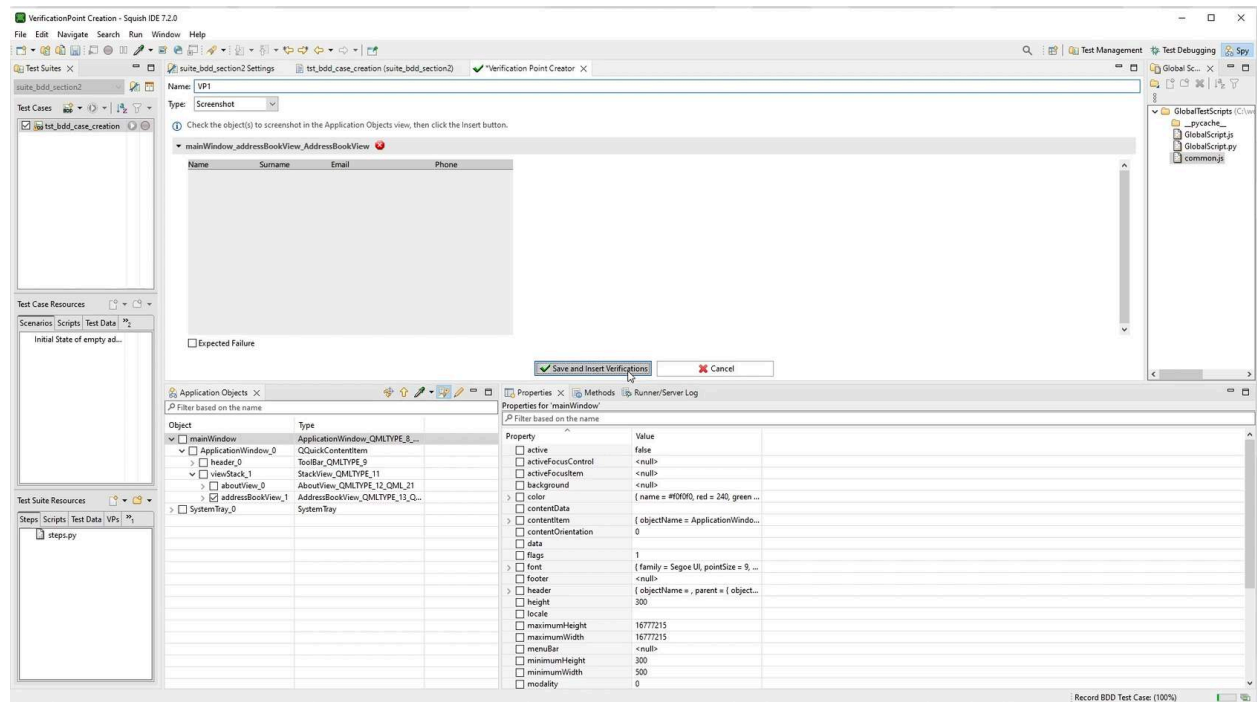
Step 5: Removing the existing entries



After running the application, we can see that two entries are here. In the test we will remove them. Removing initial entries from the address book means defining a sequence of steps:

- Verifying the application is running,
- Removing initial entries from the address book,
- Confirming the address book has zero entries.

Step 6: Recording the test steps



The recording process in Squish for BDD is done step by step, starting with verifying the application's main window is visible, then removing initial entries, and finally, performing a screenshot verification to ensure the address book is empty.

Step 7: Exploring [steps.py](#)

```
suite_bdd_section2 Settings  tst_bdd_case_creation (suite_bdd_section2)  steps.py (suite_bdd_section2) X
1  # -*- coding: utf-8 -*-
2
3  # A quick introduction to implementing scripts for BDD tests:
30
31  import names
32
33
34  @Given("quickaddressbook is running")
35  def step(context):
36      startApplication("quickaddressbook")
37      test.compare(waitForObjectExists(names.mainWindow_QQuickApplicationWindow).visible, True)
38
39  @When("I remove initial entries")
40  def step(context):
41      mouseClicked(waitForObject(names.header_Remove_ToolButton), 31, 17, Qt.LeftButton)
42      mouseClicked(waitForObject(names.header_Remove_ToolButton_2), 31, 17, Qt.LeftButton)
43
44  @Then("address book should have zero entries")
45  def step(context):
46      snooze(1) #to let animation be completed
47      test.compare(waitForObjectExists(names.addressBookView_Item).children.count(), 1)
48
```

The implementation of the steps is found in the **steps.py** file, which can be edited for further refinements, such as replacing a screenshot verification with a verification using the compare function.

Using Parameters

The `step()` functions can be more reusable if they include optional arguments in addition to context. For each additional argument added in the function signature, a corresponding reference must be written in the decorator of the function. The syntax in the decorator is `'|type|'` with `type` having to be replaced by one of the supported types:

- **any**: one or more arbitrary characters
- **integer**: positive or negative integer value
- **word**: single word that can consist of letters, digits, and underscores.

None

```
@When("I add a new person '|word|', '|word|', '|integer|', '|any|' to address book")
def step(context, forename, surname, phone, email):
```

The content of the `step()` should then be modified to use the arguments instead of hard coded values.

The **Feature** file must also be slightly modified by adding single quotes around values that should be used in the parameters.

None

```
And I add a new person 'Bob', 'Koo', '500600800', 'bob.com' to address
book
```

Example :

test.feature file

None

```
Feature: Filling of quickaddressbook
  As a user I want to include values in it

  Scenario: Stae after adding entries
    Given quickaddressbook is running
    When i remove the initial entries
```

```
        And i add a new person 'Lewis', 'Hamilton', '0000000040', 'ferrari.com' to
address book
        And i add a new person 'Lando', 'Norris', '0000000040', 'mclaren.com' to
address book
        Then verify that there is only '2' entry
```

[steps.py](#) file

```
Python
# -*- coding: utf-8 -*-

# A quick introduction to implementing scripts for BDD tests:
#
# This file contains snippets of script code to be executed as the .feature
# file is processed. See the section 'Behaviour Driven Testing' in the 'API
# Reference Manual' chapter of the Squish manual for a comprehensive reference.
#
# The decorators Given/When/Then/Step can be used to associate a script snippet
# with a pattern which is matched against the steps being executed. Optional
# table/multi-line string arguments of the step are passed via a mandatory
# 'context' parameter:
#
# @When("I enter the text")
# def whenTextEntered(context):
#     <code here>
#
# The pattern is a plain string without the leading keyword, but a couple of
# placeholders including |any|, |word| and |integer| are supported which can be
# used to extract arbitrary, alphanumeric and integer values resp. from the
# pattern; the extracted values are passed as additional arguments:
#
# @Then("I get |integer| different names")
# def namesReceived(context, numNames):
#     <code here>
#
# Instead of using a string with placeholders, a regular expression can be
# specified. In that case, make sure to set the (optional) 'regex' argument
# to True.

import names

@Given("quickaddressbook is running")
```

```

def step(context):
    startApplication("quickaddressbook")

test.compare(waitForObjectExists(names.mainWindow_QQuickApplicationWindow).visible
, True)

@When("i remove the initial entries")
def step(context):
    doubleClick(waitForObject(names.header_Remove_ToolButton), 34, 18,
Qt.LeftButton)

@Then("these should be zero entries")
def step(context):
    snooze(1)
    test.vp("VP1")

@When("i add a new person '|word|', '|word|', '|integer|', '|any|' to address
book")
def step(context, foreName, surName, phone, email):
    mouseClick(waitForObject(names.activeFocusControl_Add_ToolButton), 15, 21,
Qt.LeftButton)
    mouseClick(waitForObject(names.addViewComponent_firstNameField_TextField), 35,
6, Qt.LeftButton)
    type(waitForObject(names.addViewComponent_firstNameField_TextField), foreName)
    mouseClick(waitForObject(names.addViewComponent_lastNameField_TextField), 27,
5, Qt.ShiftModifier, Qt.LeftButton)
    type(waitForObject(names.addViewComponent_lastNameField_TextField), surName)
    mouseClick(waitForObject(names.addViewComponent_phoneNumberField_TextField),
66, 10, Qt.ShiftModifier, Qt.LeftButton)
    type(waitForObject(names.addViewComponent_phoneNumberField_TextField), phone)
    mouseClick(waitForObject(names.addViewComponent_emailAddressField_TextField),
51, 4, Qt.ShiftModifier, Qt.LeftButton)
    type(waitForObject(names.addViewComponent_emailAddressField_TextField), email)
    mouseWheel(waitForObject(names.mainWindow_addViewComponent_EditView), 200,
125, 0, -60, Qt.ShiftModifier)
    type(waitForObject(names.addViewComponent_emailAddressField_TextField),
"<Return>")
    mouseClick(waitForObject(names.header_Back_ToolButton), 21, 10,
Qt.ShiftModifier, Qt.LeftButton)

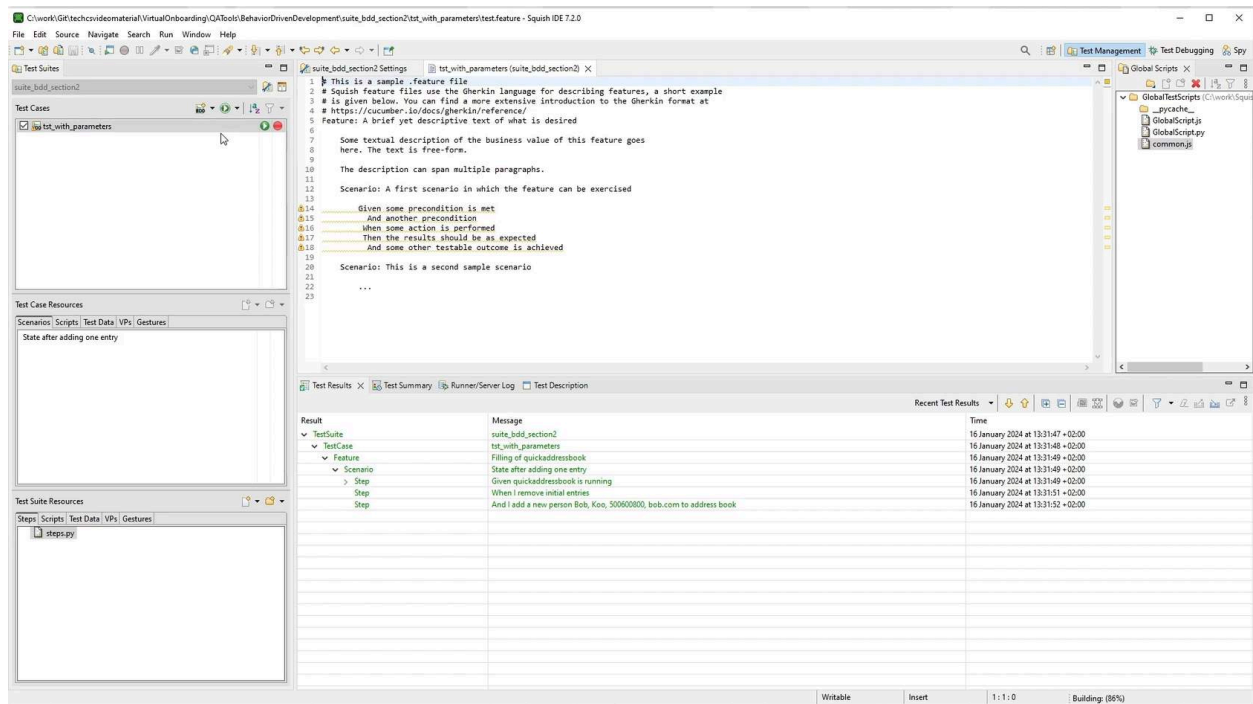
@Then("verify that there is only '|integer|' entry")
def step(context, num):
    snooze(1)

```

```
test.compare(waitForObjectExists(names.addressBookView_Item).children.count(),
num+2)
```

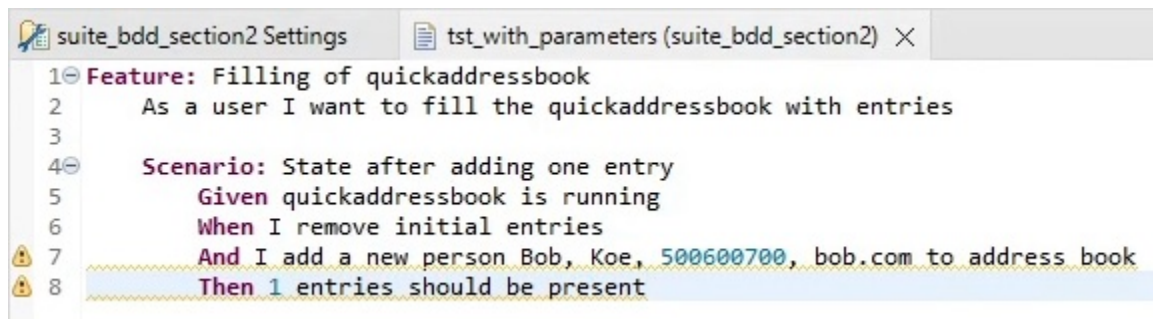
STEPS - How to Use Parameters in a Step

Step 1: Creating a New BDD Test Case



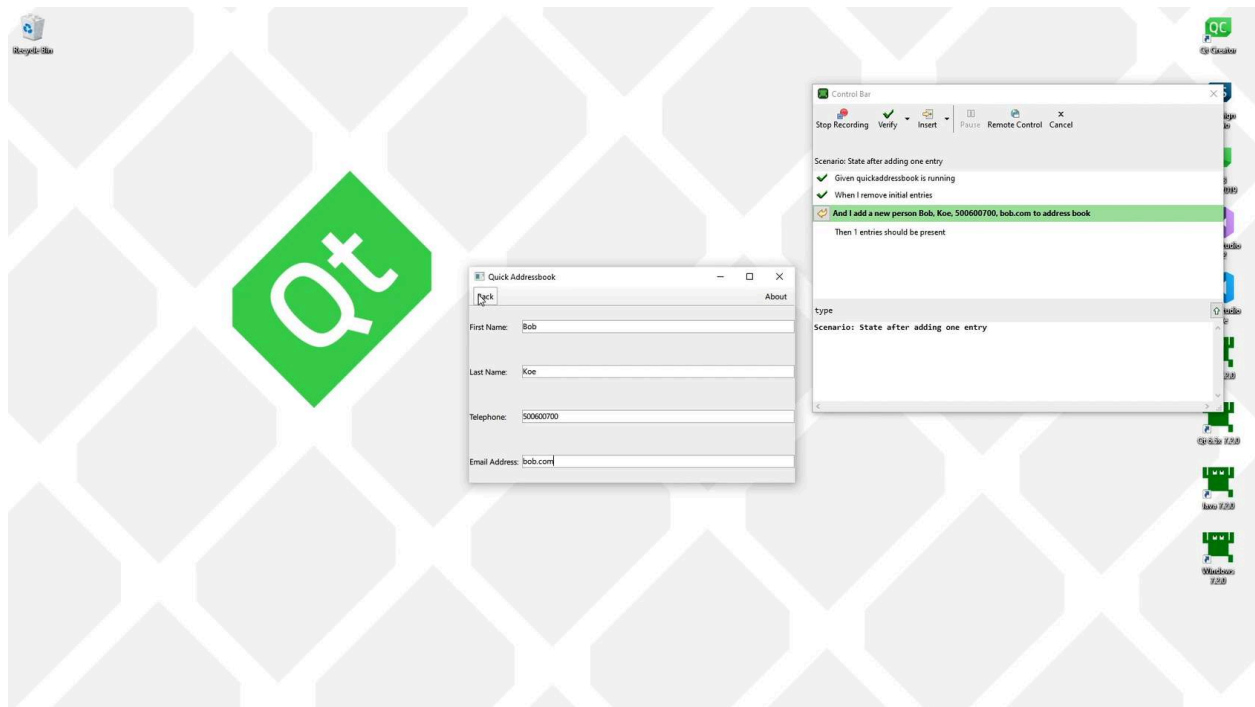
We start by creating a new BDD test case named `tst_with_parameters` in the Squish IDE, focusing on adding entries to a quickaddressbook application and verifying the number of entries present.

Step 2: Writing the Feature and Scenario



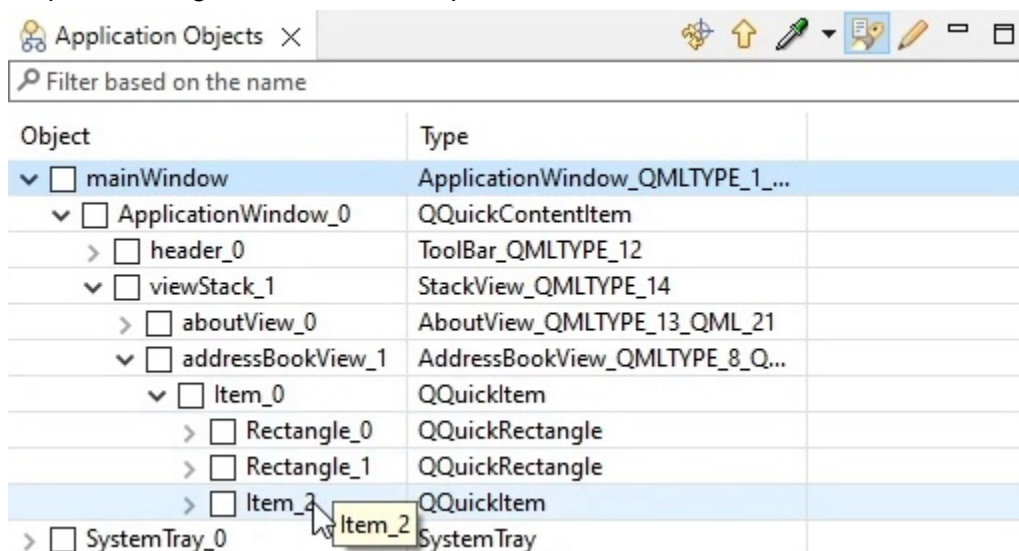
We will write a scenario that consists of adding a new entry and checking that we have the entry added. Initially, the scenario is written without parameters to add a specific person and verify the entry count.

Step 3: Recording test steps



Since we are starting from the previous test suite script, Steps already implemented in the steps.py script are reused, and new steps are recorded for actions without existing implementations. We will record a step for adding a new address book entry. We will write the last step in the script without using the recording feature. We will just check the number of children the “address book item” has, just as in the previous example.

Step 4: Utilizing Parameters in Steps



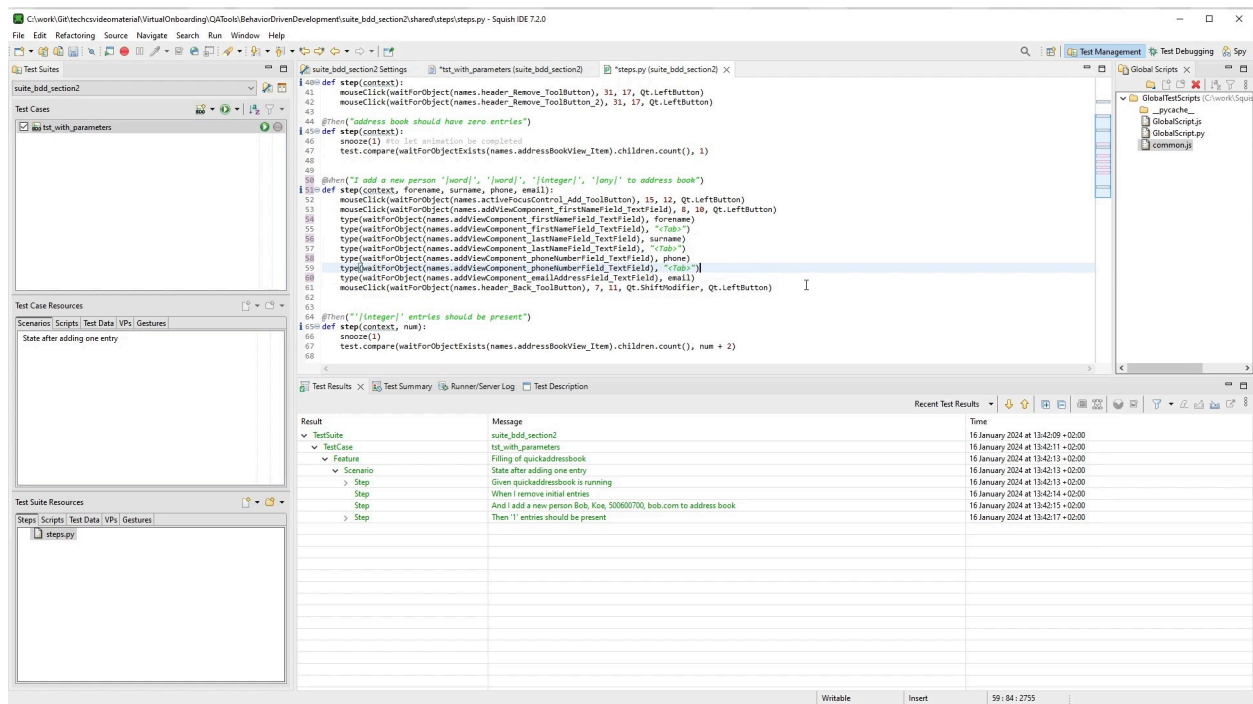
After removing an entry, explore the AUT in the object tree view. Under AddressBookView, we can see that there is an item with three children. This corresponds to the number of children when one entry is present. We can use the compare function to check this.

Step 5: Comparing multiple entries

```
63
64 @Then("'integer/' entries should be present")
65 def step(context, num):
66     snooze(1)
67     test.compare(waitForObjectExists(names.addressBookView_Item).children.count(), num + 2)
68
```

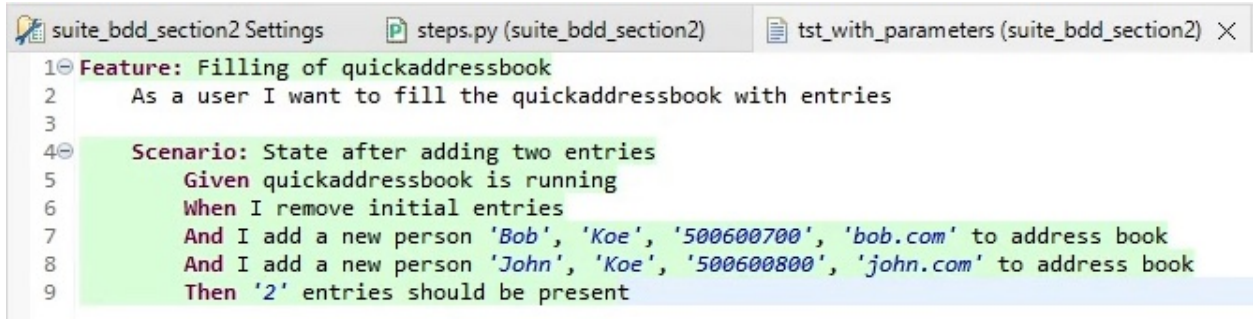
Our implementation works when we want to verify that there is one entry. But when there is more than one entry, we need to modify the step function. We will rewrite the decorator. We need to use a placeholder with the type for the parameter to use.

Step 6: Adding more entries



We add single quotes between the parameters in the two steps where we add the persons, like before, we will need to modify the decorator first by using parameter types. We will add single quotes and vertical bars between the parameters type. We will also change the function signature and pass through the arguments.

Step 7: Changing the parameter to two



```
suite_bdd_section2 Settings  steps.py (suite_bdd_section2)  tst_with_parameters (suite_bdd_section2) X
1 Feature: Filling of quickaddressbook
2   As a user I want to fill the quickaddressbook with entries
3
4 Scenario: State after adding two entries
5   Given quickaddressbook is running
6   When I remove initial entries
7   And I add a new person 'Bob', 'Koe', '500600700', 'bob.com' to address book
8   And I add a new person 'John', 'Koe', '500600800', 'john.com' to address book
9   Then '2' entries should be present
```

Since at the end of the scenario we will have two entries, we need to change the parameter to '2' in the last step.

Using Tables

Tables of data can also be passed into the steps. This is useful when a list of values is used in a step.

The table can be created directly in the `.feature` file. In that case, the table is inserted after the step description. The rows are written on different lines and the columns are separated with "|":

None

```
And I add new persons to address book
  | forename | surname | email | phone |
  | John | Smith | john@m.com | 123123 |
  | Alice | Thomson | alice@m.com | 234234 |
```

In the `step()` function, a for loop can be used to loop through the rows. The `context.table` property is used to access the table in the script. As a column header can also be written in the **Feature** file, and all the table contents are passed to the script the first row may need to be dropped.

None

```
@Step("I add new persons to address book")
def step(context):
    table = context.table
    # Drop initial row with column headers
    table.pop(0)
    for (forename, surname, phone, email) in table:
```

If it is more practical to store table data in an external file, the **From** keyword followed by the file path relative to the `.feature` file can be used to read the table data. The data file must exist and be saved in one of the following formats.:

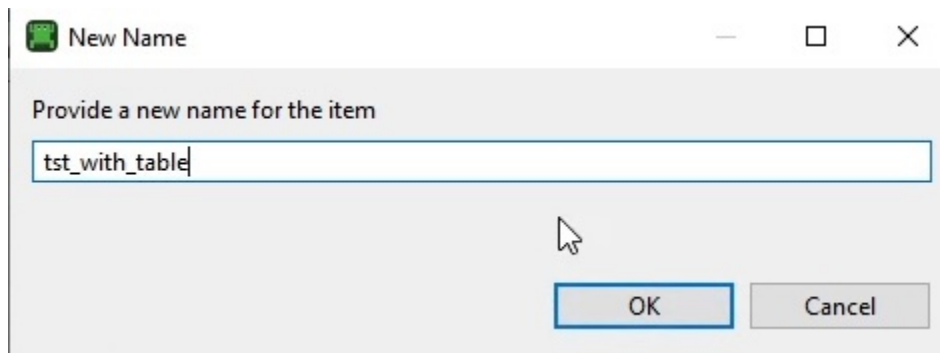
- .txt or no extension
- .csv
- .tsv
- .xls

None

And I add new persons to address book
From testdata/contact_list.tsv

STEPS - How to Use Tables in a Step

Step 1: Repurposing a test case



We rename the existing test case from suite_bdd_section3 to incorporate data-driven testing by adding multiple entries to an address book using a table format.

Step 2: Structuring the feature file

```
3
4 #Steps for new entry
5 Scenario: State after adding entries
6     Given quickaddressbook application is running
7     When I remove initial entries
8     And I add new persons to address book
9         | forename | surname | phone | email |
10        | John | Smith | 123123 | john@m.com |
11        | Alice | Thomson | 234234 | alice@m.com |
12     Then '2' entries should be present
13
```

The feature file's scenario "State after adding entries" is updated to remove individual entry steps in favor of a table structure that lists multiple entries, each with columns for forename, surname, phone, and email. Each set of data is written in one line and each cell is separated by a vertical bar.

Step 3: Implementing the table in test steps

```

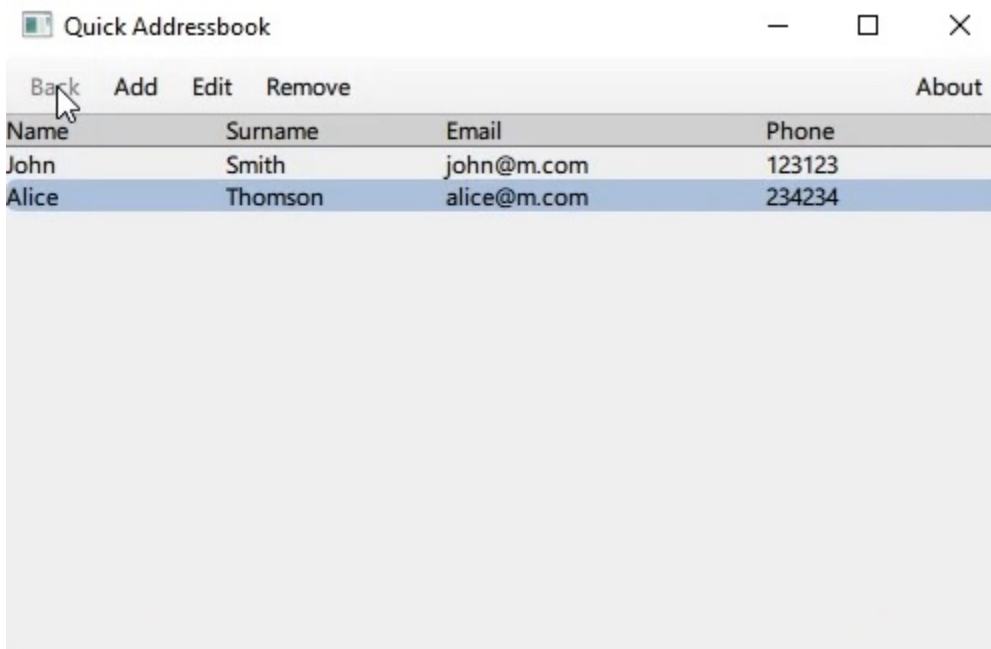
81
82 @when("I add new persons to address book")
83 def step(context):
84     table = context.table
85     table.pop(0)
86     for (forename, surname, phone, email) in table:
87         type(waitForObject(names.mainWindow_QQuickApplicationWindow), "<Tab>")
88         mouseClicked(waitForObject(names.activeFocusControl_Add_ToolButton))
89         mouseClicked(waitForObject(names.addViewComponent_firstNameField_TextField))
90         type(waitForObject(names.addViewComponent_firstNameField_TextField), forename)
91         type(waitForObject(names.addViewComponent_firstNameField_TextField), "<Tab>")
92         type(waitForObject(names.addViewComponent_lastNameField_TextField), surname)
93         type(waitForObject(names.addViewComponent_lastNameField_TextField), "<Tab>")
94         type(waitForObject(names.addViewComponent_phoneNumberField_TextField), phone)
95         type(waitForObject(names.addViewComponent_phoneNumberField_TextField), "<Tab>")
96         type(waitForObject(names.addViewComponent_emailAddressField_TextField), email)
97         mouseClicked(waitForObject(names.header_Back_ToolButton))

```

Found at: steps

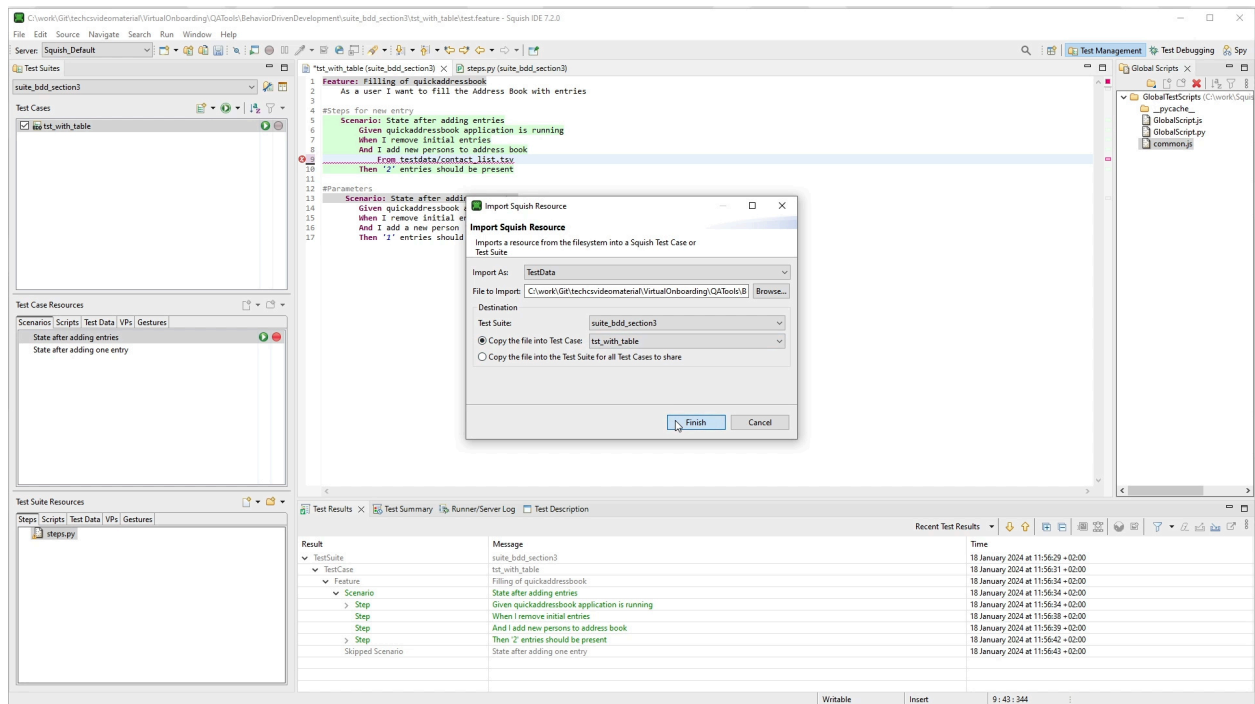
After creating the table structure in the feature file, we use the "Create Missing Steps Implementation" option in Squish to generate a function that handles the table data. The context.table property is utilized to process this data in the steps.py script. We need to remove the header row and then loop through each row of data, we can just copy-paste the implementation from the step for adding a single person.

Step 4: Running the test



Upon executing the test, we successfully add the specified entries to the address book, demonstrating the effectiveness of using tables for data-driven testing in BDD with Squish.

Step 5: Using an external file



Instead of embedding a data table directly in the feature file, we use the "From" keyword followed by the path to an external data file. This method supports various file formats, including text files, Excel files, and .csv/.tsv files, offering versatility in how test data can be structured and stored.

Sharing data between steps and scenarios

Data can be shared between steps and scenarios with the **context.userData** property. All data stored in **context.userData** can be accessed in all steps and Hooks in all scenarios of a given feature.

Here data is stored in **context.userData** in one step:

None

```
@Then("I add a new person '|word|', '|word|', '|integer|', '|any|'
to address book")
def step(context, forename, surname, phone, email):
    context.userData = {}
    context.userData['forename'] = forename
    context.userData['surname'] = surname
```

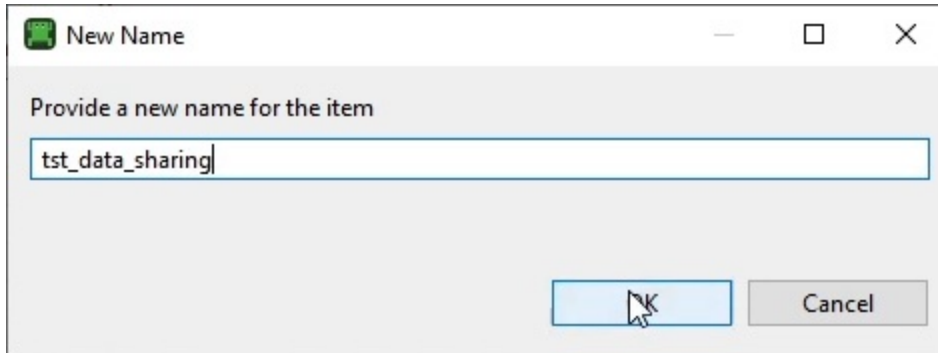
Then, the stored data can be accessed and modified in another:

None

```
@Then("previously entered forename and surname are in the table")
def step(context):
    test.ocrTextPresent(context.userData['forename'], {
    "tesseract":
    { "psm": 3 } }, waitForObjectExists(names.mainWindow))
    test.ocrTextPresent(context.userData['surname'], { "tesseract":
    { "psm": 3 } }, waitForObjectExists(names.mainWindow))
```

STEPS - How to Share Data Between Steps

Step 1: Repurposing a test case



We explore data sharing between steps in a test case named `tst_data_sharing` using the Qt Quick Address Book example. The goal is to add an entry and verify its details in subsequent steps. We rename the existing test case from `suite_bdd_section3`.

Step 2: Modifying the test scenario

```
*tst_data_sharing (suite_bdd_section3) X
1 Feature: Filling of quickaddressbook
2   As a user I want to fill the Address Book with entries
3
4 #Steps for new entry
5 Scenario: Forename and surname are added to the table
6   Given quickaddressbook application is running
7   When I add a new person 'John', 'Doe', '500600700', 'john@m.com' to address book
8   Then previously entered forename and surname are in the table
9
```

The test scenario is updated to add a new person to the address book and verify the forename and surname in the table. This demonstrates the need for data sharing between the "When" and "Then" steps - The entry needs to be accessible in both steps to verify using OCR.

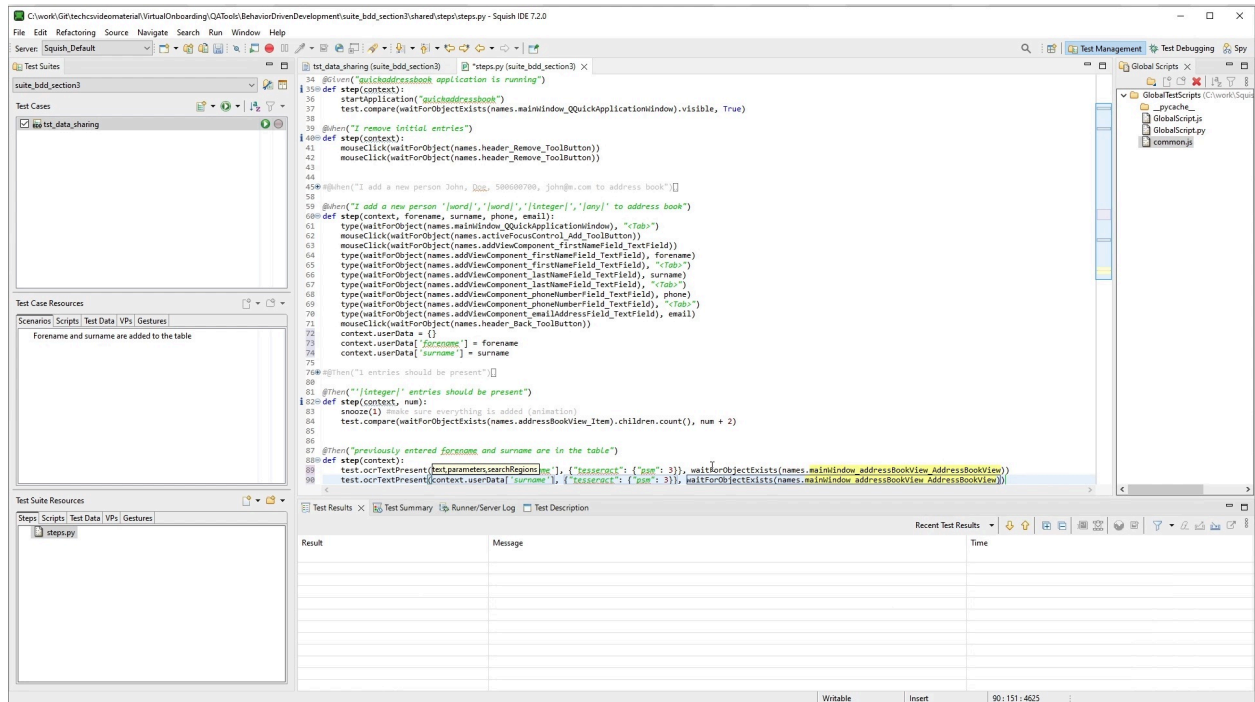
Step 3: Implementing data sharing

```
58
59 @When("I add a new person '/word/', '/word/', '/integer/', '/any/' to address book")
60 def step(context, forename, surname, phone, email):
61     type(waitForObject(names.mainWindow_QQuickApplicationWindow), "<Tab>")
62     mouseClick(waitForObject(names.activeFocusControl_Add_ToolButton))
63     mouseClick(waitForObject(names.addViewComponent_firstNameField_TextField))
64     type(waitForObject(names.addViewComponent_firstNameField_TextField), forename)
65     type(waitForObject(names.addViewComponent_firstNameField_TextField), "<Tab>")
66     type(waitForObject(names.addViewComponent_lastNameField_TextField), surname)
67     type(waitForObject(names.addViewComponent_lastNameField_TextField), "<Tab>")
68     type(waitForObject(names.addViewComponent_phoneNumberField_TextField), phone)
69     type(waitForObject(names.addViewComponent_phoneNumberField_TextField), "<Tab>")
70     type(waitForObject(names.addViewComponent_emailAddressField_TextField), email)
71     mouseClick(waitForObject(names.header_Back_ToolButton))
72     context.userData = {}
73     context.userData['forename'] = forename
74     context.userData['surname'] = surname
75
```

We use the "Create Missing Steps Implementation" option in Squish to generate a function for the "Then" step. We use the `context.userData` property in the `steps.py` file. This property allows

us to store and retrieve data across different steps within the same scenario. The `userData` is never cleared unless done so explicitly, so we clear it first before writing data.

Step 4: Verifying data between steps



For verification, OCR text verification checks are implemented to confirm the forename and surname stored in `context.userData` are displayed in the address book, showcasing successful data sharing and verification.

Step 5: Running the test

Test Results	
Test Summary	
Runner/Server Log	
Test Description	
Result	Message
TestSuite	suite_bdd_section3
TestCase	tst_data_sharing
Feature	Filling of quickaddressbook
Scenario	Forename and surname are added to the table
Step	Given quickaddressbook application is running
Step	When I add a new person 'John','Doe','500600700','john@m.com' to address book
Step	Then previously entered forename and surname are in the table
Pass	Verified
Detail	The phrase 'John', occurrence: 1 was found
Pass	Verified
Detail	The phrase 'Doe', occurrence: 1 was found
Pass	Pass

Sharing data between steps in BDD testing with Squish, enhancing the flexibility and efficiency of test scripts by allowing for complex test scenarios that require data continuity. Running the test confirms the data is the same between the steps.

Running a scenario multiple times, with Scenario Outline

In addition to iterating and running the same step with different data values, it is possible to run the same scenarios multiple times but with different test data.

For this the keywords Scenario Outline and Examples are used:

- **Scenario Outline** : describes the scenario template that will be run multiple times
 - The placeholder that will be replaced with different values are marked using '< >'.
- **Examples** is the data table which contains the values for the placeholders.
 - It can be optionally stored in an external file.

None

Scenario Outline: Adding a single entry multiple times

Given quickaddressbook application is running

When I remove initial entries

And I add a new person

'<forename>', '<lastname>', '<phone>', '<email>' to address book

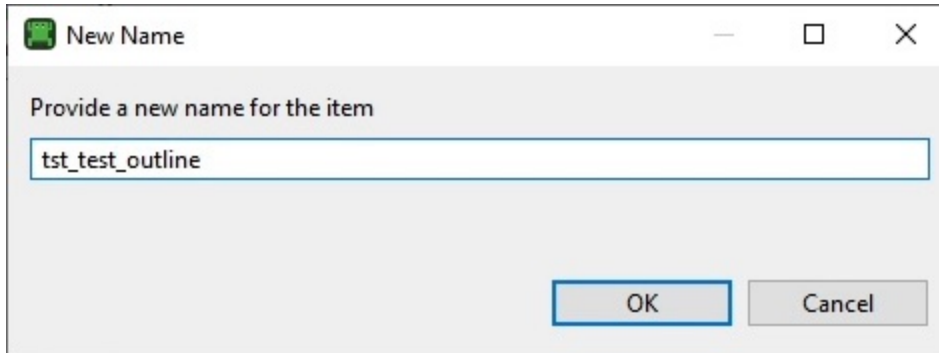
Then '1' entries should be present

Examples:

	forename		lastname		email		phone	
	John		Doe		john@m.com		500600700	
	Bob		Koo		bob@m.com		500600800	

STEPS - How to Use Scenario Outline Keyword to Run the Same Scenario with Different Data Sets

Step 1: Repurposing a test case



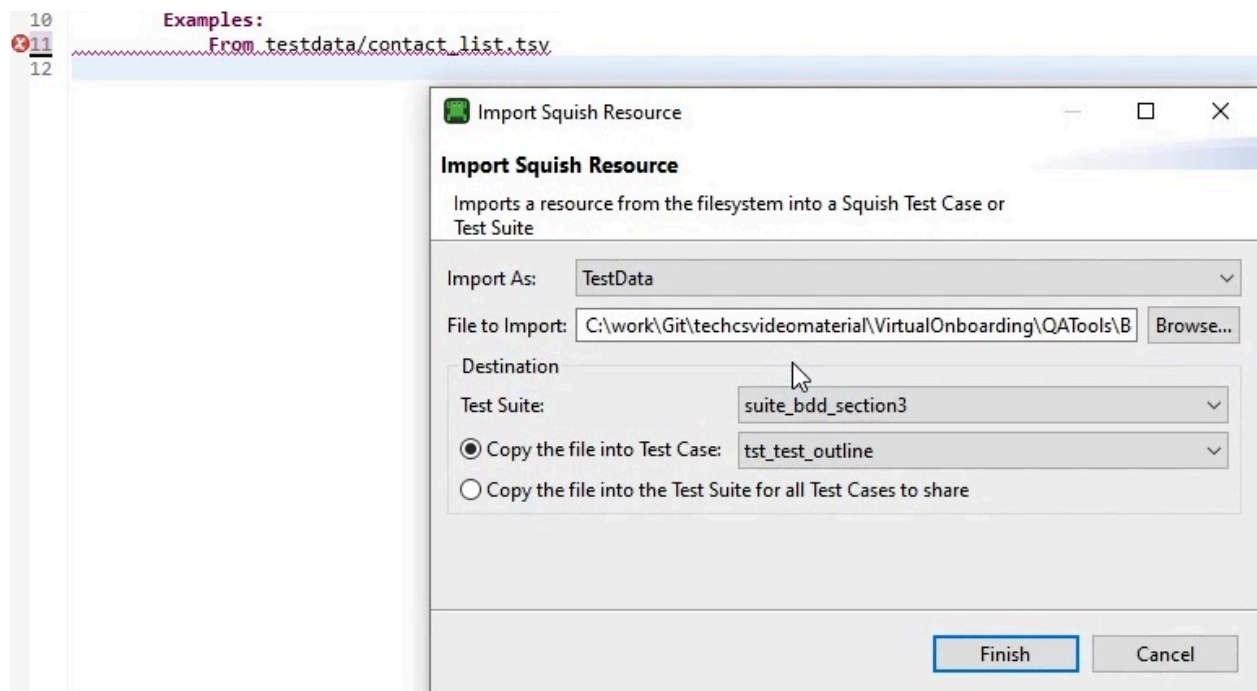
We rename the existing test case from `suite_bdd_section3` to implement scenario outlines. The "Scenario Outline" feature in Squish is utilized to streamline the process of running a scenario multiple times with different data sets, eliminating the need for repetitive scenario duplication.

Step 2: Utilizing scenario outlines

```
3
4 #Steps for new entry
5 Scenario Outline: Adding a single entry multiple times
6     Given quickaddressbook application is running
7     When I remove initial entries
8     And I add a new person '<forename>','<surname>','<phone>','<email>' to address book
9     Then '1' entries should be present
10    Examples:
11        | forename | surname | phone | email |
12        | John   | Doe    | 123123 | john@m.com |
13        | Bob    | Koo    | 234234 | bob@m.com |
14
```

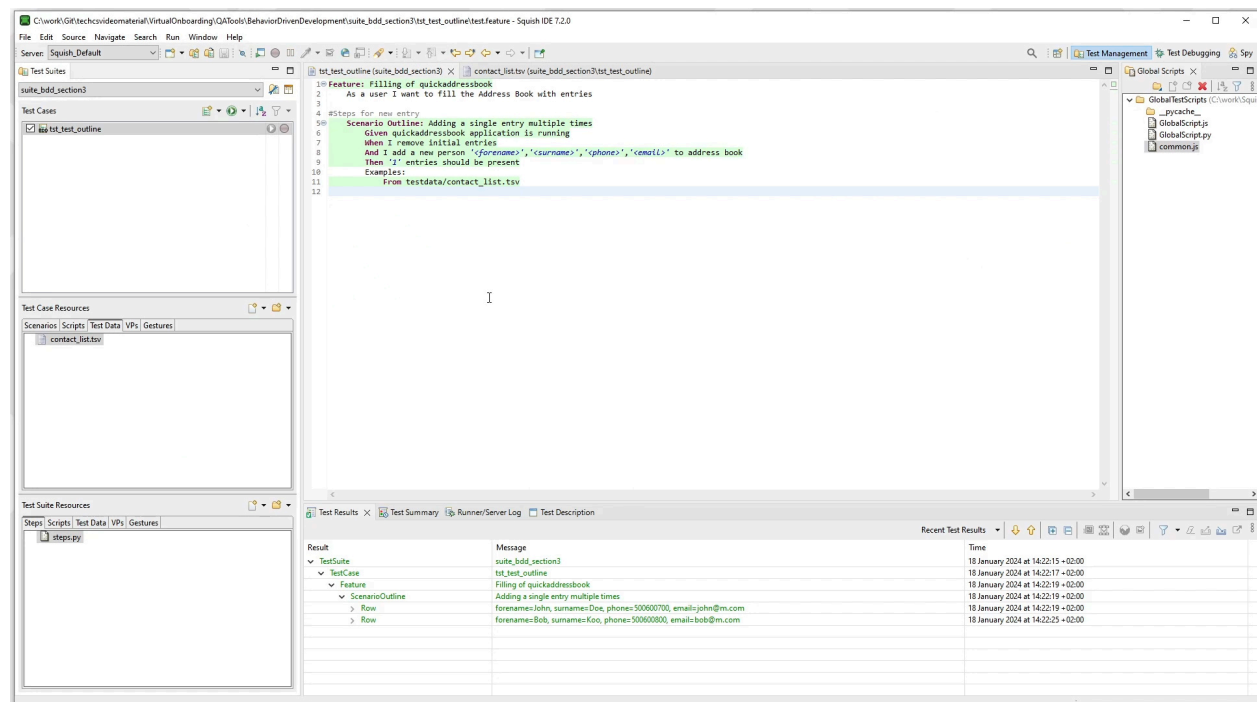
Changing the scenario definition to "Scenario Outline" and introducing the "Examples" keyword, we can define a table of data that represents different sets of parameters to be used in the test. We use `<>` notation (e.g., `<forename>`), indicating where the data from the "Examples" table should be inserted during test execution.

Step 3: Using external test data



For enhanced manageability, data can also be externalized to a file using the "From" keyword, followed by the path to a data file (e.g., CSV, TSV), which is then imported into the test suite.

Step 4: Running the test



Running the test showcases the scenario being executed multiple times with the provided sets of data, demonstrating the effectiveness and efficiency of using Scenario Outlines and external data files in BDD testing with Squish without duplicating the scenario without copy-pasting scenarios and test data.

Structure of a BDD Test Case in Squish

Like for script-based testing, for BDD tests, Squish will execute the test (.py, .js,...) file. By default, this script does the following 👍

1. Loading of the file containing Squish BDD-dedicated API.
2. Setup of the hooks to perform actions during test execution.
3. Search and import of step definitions that are found in the directories specified.
4. Execution of all the scenarios in a specified Gherkin .feature file.

[Hook functions](#) are used to trigger extra functionalities as a .feature file is run. They are executed when a feature/scenario/step starts or ends. It is common that some actions need to be performed before or after some event occurs.

For instance, hook functions are handy to use when:

- Global variables should be initialized before test execution starts.
- An application needs to be started before a Feature is executed.
- Temporary files should be removed after a Scenario is executed.

hook() functions can be associated with any of the OnFeatureStart/OnFeatureEnd/OnScenarioStart/OnScenarioEnd/OnStepStart/OnStepEnd events, and several hook() functions can be created for the same event. This is done by writing a hook() function followed by a decorator with the name of the event. For instance, a common use-case is to use the OnScenarioStart/OnScenarioEnd hooks to start and stop an AUT:

None

```
@OnScenarioStart
```

```
def hook(context):
```

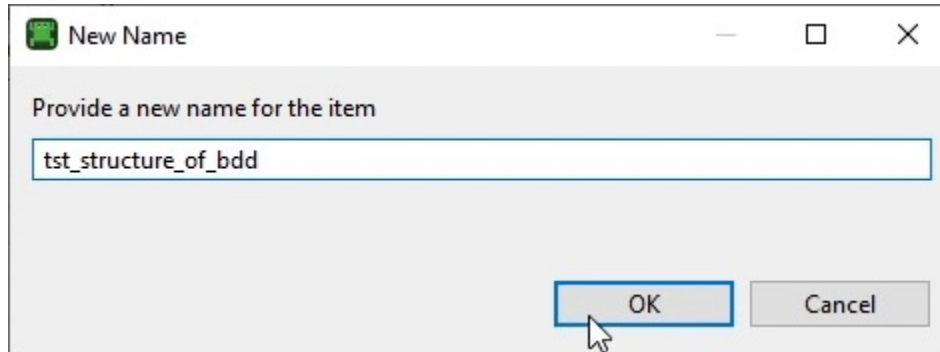
```
    startApplication("quickaddressbook")
```

The BDD context singleton objects provides properties such as 👍

- **context.title** - name of the current scenario
 - **context.text** - name of the current step
 - **context.multiLineText** - multi-line text if being passed into the step
-

STEPS - Structure of BDD Tests in Squish

Step 1: Repurposing a test case



We will be starting from the test suite `suite_bdd_section2` in this example. We rename the existing test case to `tst_structure_of_bdd`. In Squish, BDD tests start with creating a test case, leading to the generation of a feature file for describing BDD tests and a `test.py` script for execution.

Step 2: Test case and feature file

```
test.py (suite_bdd_section2\tst_structure_of_bdd) X
1 source(findFile('scripts', 'python/bdd.py'))
2
3 setupHooks('../shared/scripts/bdd_hooks.py')
4 collectStepDefinitions(['./steps', '../shared/steps'])
5
6 def main():
7     testSettings.throwOnFailure = True
8     runFeatureFile('test.feature')
9
```

The `test.py` script includes loading the Squish BDD API, setting up hooks for additional functionalities, and importing step definitions, usually from a `steps.py` file. It concludes by executing scenarios defined in the feature file.

Step 3: Adding hooks for scenario and features

```

23
24 @OnFeatureStart
25 def hook(context):
26     test.log("Feature starts: " + context.title)
27     global counter
28     counter = 0
29
30 @OnFeatureEnd
31 def hook(context):
32     global counter
33     test.log(str(counter) + " steps executed")
34     test.log("Feature ends: " + context.title)
35
36 @OnScenarioStart
37 def hook(context):
38     test.log("Scenario starts: " + context.title)
39     startApplication("quickaddressbook")
40
41 @OnScenarioEnd
42 def hook(context):
43     for ctx in applicationContextList():
44         ctx.detach()
45     test.log("Scenario ends: " + context.title)
46

```

Hooks in `bdd_hooks.py` allow triggering functionalities at the start or end of features, scenarios, or steps, enhancing test flow control. Each `hook()` function has a parameter called `context`. They can log scenario titles, manage application startup/shutdown, and track executed steps. We will write hooks for when the scenario and features start and ends.

Step 4: Changing the starting of the application

```

34 @Given("quickaddressbook application is running")
35 def step(context):
36     startApplication("quickaddressbook")
37     test.compare(waitForObjectExists(names.mainWindow_QQuickApplicationWindow).visible, True)
38

```

we just need to remove the line to start the application in the `steps.py` since we are starting the AUT with a hook.

Step 5: Running the test

Test Results × Test Summary Runner/Server Log Test Description	
Result	Message
▼ TestSuite	suite_bdd_section2
▼ TestCase	tst_structure_of_bdd
▼ Feature	Filling of quickaddressbook
Log	Feature starts: Filling of quickaddressbook
▼ Scenario	Initial state of empty address book
Log	Scenario starts: Initial state of empty address book
> Step	Given quickaddressbook application is running
Step	When I remove initial entries
> Step	Then Address book should have zero entry
Log	Scenario ends: Initial state of empty address book
Log	0 steps executed

After running the test and we can see the counter doesn't increase as expected, we need to add a hook executed when a step starts. We will just write a function to increase the counter.

Step 6: Step hooks

```

47 @OnStepStart
48 def hook(context):
49     test.log("Step starts: " + context.text)
50     global counter
51     counter += 1
52
53 @OnStepEnd
54 def hook(context):
55     test.log("Step ended: " + context.text)

```

To address the issue of the non-increasing counter, we implement the `@OnStepStart` hook to increment a global counter each time a step begins, and log the step's text. A corresponding `@OnStepEnd` hook is added to log the conclusion of each step.

Test Execution and Debugging

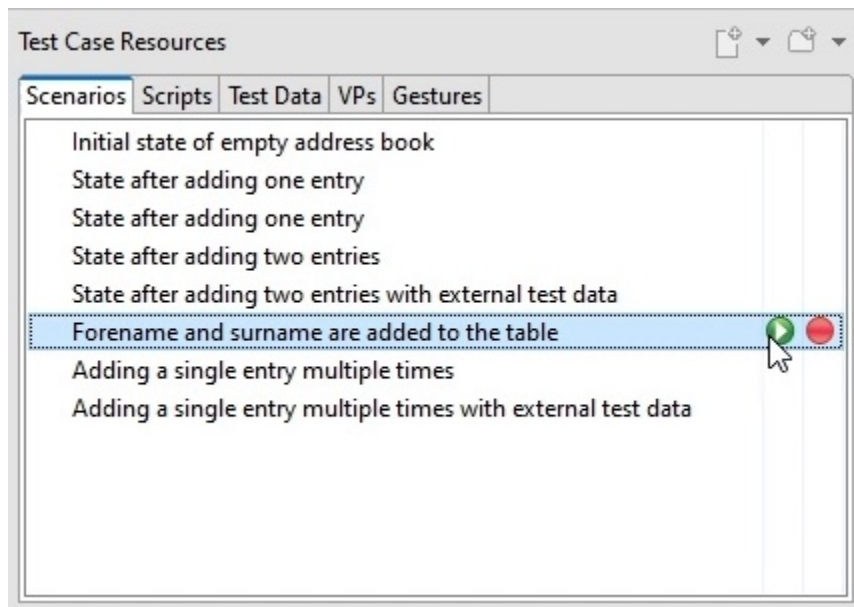
After a Scenario is executed, the Feature file is colored according to the execution results. Detailed information is also available in the Test Results View.

In case of error or to inspect the code, breakpoints can be added both in the Feature file at any line containing a step or at any line of executed code in steps script file. Squish will switch to Test Debugging mode when reaching a breakpoint. Like with tests based on script test case, it is possible to:

- Check script variables
- Spy application objects to inspect them and their properties
- Run custom code in the Squish Script Console
- If a breakpoint is placed at a step definition or a hook is reached, add Verification Points or record code snippets.

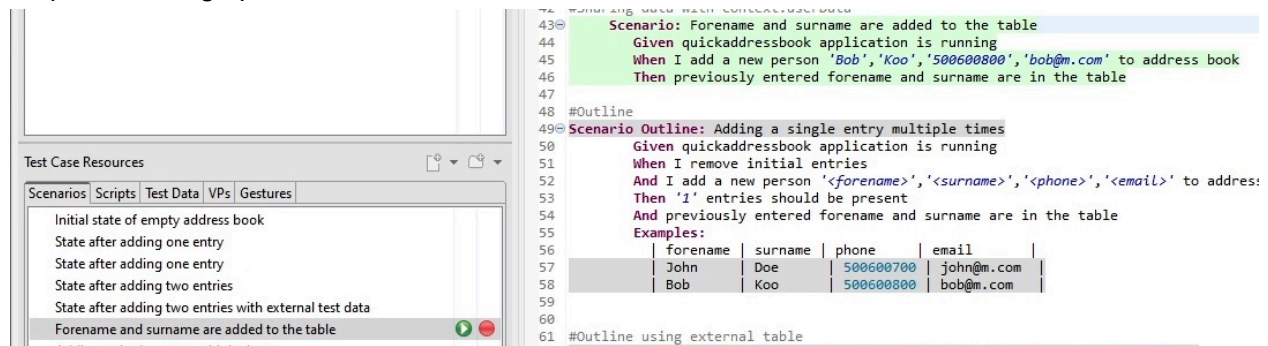
STEPS - Test Execution and Debugging Features in the Squish IDE

Step 1: Opening the test case



We will use the Qt Quick Address Book example. The application can be found in the examples subdirectory in the Squish installation. We will be starting from the test suite called `suite_bdd_section8` in this example. First, we can see we can run a specific scenario in a step file.

Step 2: Running specific scenarios



In Squish, you can run specific scenarios directly from the Test Case Resources. Selecting and running a scenario like "Forename and surname is added to table" shows successful steps in green and failed ones in red in the Test Results view.

Step 3: adding a typo

```
62 context.userData = {}
63 context.userData['forname'] = forename
64 context.userData['surname'] = surname
```

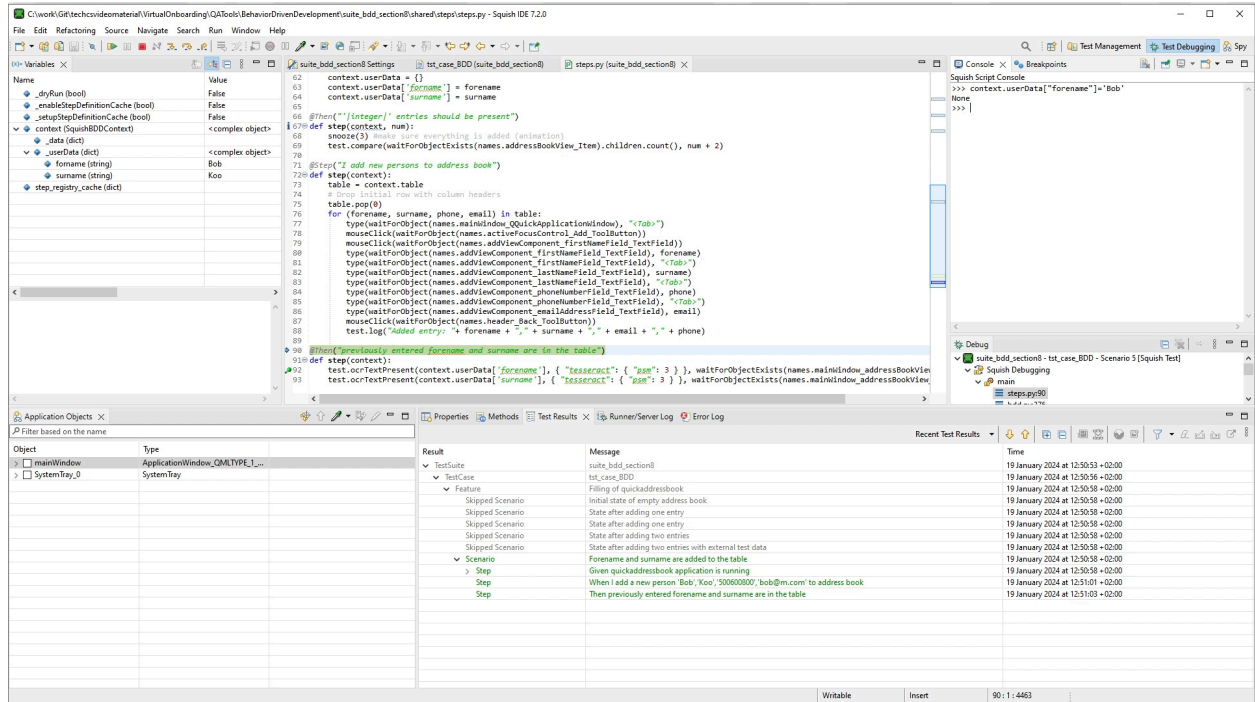
To demonstrate debugging, we add an intentional typo in the steps.py file at line 63 - changing "forename" to "forname". Running the feature file then highlights scenarios that fail due to the typo.

Step 4: setting breakpoints

```
42 #Sharing data with context.userData
43 Scenario: Forename and surname are added to the table
44     Given quickaddressbook application is running
45     When I add a new person 'Bob', 'Koo', '500600800', 'bob@m.com' to address book
46     Then previously entered forename and surname are in the table
47
```

At the end of the test execution, we can see some scenarios have failed. The error arose in scenarios that include the step "previously entered forename and surname are in the table". Breakpoints can be added both in the Python script and the feature file for detailed debugging. We will add one breakpoint at line 46 of the feature file and another one in the steps.py at line 92. Re-running a failed scenario allows you to hit these breakpoints and examine the execution.

Step 5: debugging



During debugging, we can inspect variables in the Variables view to pinpoint the typo issue. We can test by correcting the typo directly in the Console and proceeding to the next breakpoint confirms the fix, as indicated by the test turning green.

Step 6: fixing the implementation

Name	Value
_dryRun (bool)	False
_enableStepDefinitionCache (bool)	False
_setupStepDefinitionCache (bool)	False
name (string)	userData
self (SquishBDDContext)	<complex object>
step_registry_cache (dict)	

Here we can see the context object has clearly been modified. It is named self in the Variables view. Now our test succeeded. If we go back to our feature file, we can see it is green now. We now know what the issue was and could modify the implementation code.

Migration to BDD

Existing script-based tests can be extended or migrated to BDD testing. There are two possibilities:

1. Keeping any existing script-based test and extend the test suite by adding new BDD tests.
2. Converting the existing script-based tests to BDD tests.

Let's explore these in detail.

Scenario 1

Keeping existing script-based test and extend test suite by adding new BDD tests

- Squish supports test suites containing both script-based tests and BDD tests.
- If the existing script-based tests make use of a library and shared functions are called to interact with the AUT, those functions can also be used in the newly created BDD test.

Scenario 2

Converting the existing script-based tests to BDD tests

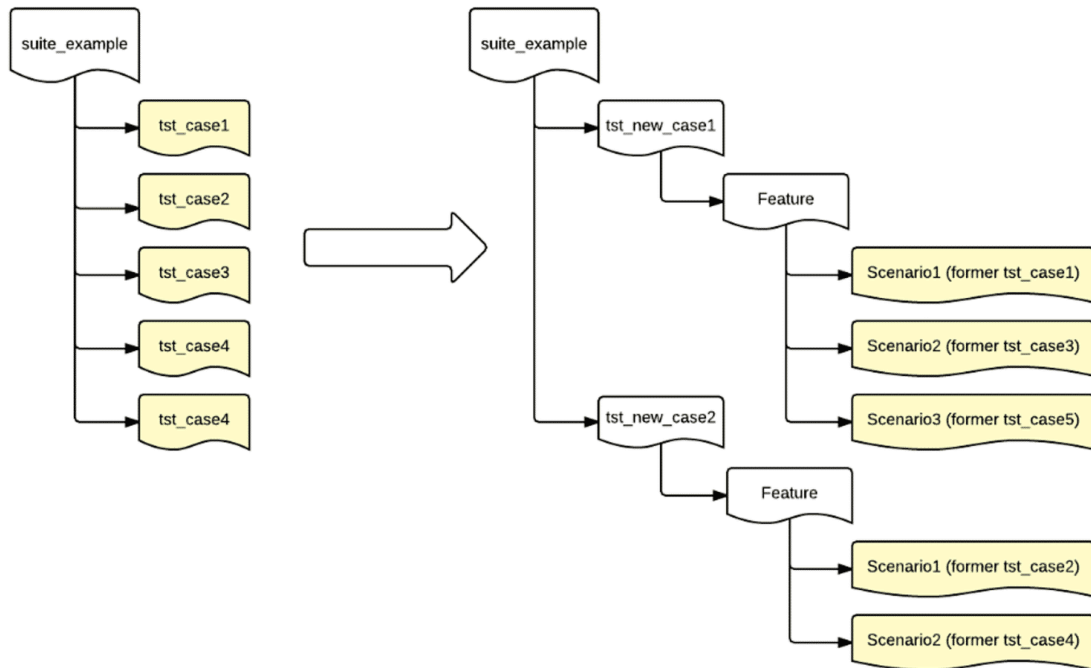
- The first step is to review all script-based tests and to group them by the feature they test.
- Each script-based test is then transformed into a scenario, which is a part of a feature.
- Once a test has been translated into a scenario, a skeleton of the missing step definitions can be generated automatically.
- This is done by right-clicking on the Scenario in the .feature file and selecting Create Missing Step Implementations.
- The functionality adds step() functions with a warning saying that the test needs to be implemented

None

```
@Given("Step whose implementation is missing")
```

```
def step(context):
```

```
    test.warning("TODO implement Step whose implementation is  
missing ")
```



A default BDD test case consists of:

1. A test (.py,.js, ...) based on the scripting language used in the test suite) that drives the execution of the text.
2. A test.feature feature file written in DSL.
3. Step implementations. Shared by the entire test suite and located in shared/steps/steps (.py, .js, ...)

Summary

Creation and Structure of BDD Tests

The journey into Behavior-Driven Development (BDD) with Squish begins with creating a test case, which generates essential files such as **test.py** for script execution and **test.feature** for describing behaviors in a structured language. This setup lays the groundwork for detailed and understandable test scenarios.

Parameterization and Data-Driven Testing

Implementing step parameters and utilizing tables significantly amplifies test versatility. Parameters allow for dynamic data insertion into steps, while tables enable comprehensive data-driven testing, making it easier to test various scenarios without altering the core test logic.

Sharing Data and Scenario Outlines

Sharing data between steps showcases Squish's capability to maintain state across different test phases, crucial for complex testing scenarios. The use of Scenario Outlines, coupled with the Examples section, streamlines executing repetitive scenarios with varied datasets, enhancing test coverage.

Execution, Debugging, and Hooks

The final stages of BDD testing with Squish emphasize test execution, debugging, and the strategic use of hooks for setup and teardown processes. These elements together ensure that tests not only run smoothly but also provide detailed feedback for troubleshooting and verifying application behavior under different conditions.